#1 Start at the index passed as argument (by default at the first element of the array A)
#2 Leaves have no children, so can't be pushed down anymore. `firstLeafIndex` returns the index of the first leaf in the heap: if `D` is the branching factor, and array's indices starting at zero, then it's `(|pairs| - 2) / D + 1`
#3 We need to identify current node's children with the highest priority, because that will be the only element that is safe to move up without breaking heap properties
#5 If the highest priority child has higher priority than the current element `p`...
#6 ... then swaps current element with the one among its children with highest priority
#7 Otherwise, the heap properties have been reinstated, and we can break out and exit the function

---

This method's running time, while asymptotically equivalent to `bubbleUp`'s, requires a slightly larger number of comparisons: in the worst case, `D * logD(n)`, for a heap with branching factor `D` and containing `n` elements. This is also the reason for which, as we will see in section 2.10, increasing the branching factor (the max number of children for each node) indefinitely is not a good idea.
